==========

==========

CHAPTER 27 — COMMON MMU BUGS & FAILURE MODES

==========

==========

A maximum-detail, RTOS/QNX-style breakdown of the most frequent MMU failures seen in real systems, how they manifest, how to diagnose them, and how to fix them.

This chapter is extremely practical — it covers **the bugs that actually happen in production**, the ones that kernel engineers and hypervisor developers spend weeks chasing.

****

****

27.1 — Why MMU Bugs Are Hard

****

****

MMU bugs are uniquely painful because:

- symptoms appear far from the root cause
- failures are nondeterministic
- SMP cores behave differently
- stale TLB entries hide descriptor changes
- walk-cache retains old table pointers
- caches obscure memory writes
- device memory behaves differently
- Stage-2 faults propagate differently

This chapter gives you **the real failure modes** and how to diagnose them.

****

****

27.2 — Failure Mode #1: Wrong Memory Attributes

****

****

Symptom

- device malfunction
- corrupted registers
- random crashes
- DMA corruption
- unpredictable timing

Cause

MMIO mapped as **Normal WB** instead of **Device-nGnRnE**.

Diagnosis

Page-table dump shows:

```
AttrIndx = 0 (Normal WB)
```

Fix

```
AttrIndx = 2 (Device-nGnRnE)
SH = 00 (Non-Shareable)
PXN = 1
UXN = 1
```

****

****

27.3 — Failure Mode #2: Wrong Shareability

****

****

Symptom

- SMP cores see different data
- nondeterministic crashes
- stale data on one core
- race conditions that “shouldn’t happen”

Cause

RAM mapped as **Non-Shareable**.

Diagnosis

Descriptor shows:

```
SH = 00 (Non-Shareable)
```

Fix

```
SH = 11 (Inner Shareable)
```

****

****

27.4 — Failure Mode #3: Wrong Permissions

****

****

Symptom

- page faults
- execute faults
- permission faults
- kernel crashes
- user processes crash randomly

Cause

Incorrect AP/PXN/UXN bits.

Examples

- data mapped executable
- code mapped writable
- user memory mapped PXN
- kernel memory mapped UXN

Fix

Enforce W^X:

```
Code: RO + executable
Data: RW + non-exec
Stack: RW + non-exec
MMIO: RW + non-exec
```

****

****

27.5 — Failure Mode #4: Missing Access Flag (AF)

****

****

Symptom

- page faults on first access

- random crashes
- faults only on cold pages

Cause

AF bit = 0.

Diagnosis

Descriptor:

`AF = 0`

Fix

Set AF = 1.

****

****

27.6 — Failure Mode #5: Wrong Granule Size

****

****

Symptom

- MMU fails to enable
- page tables unreadable
- translation faults at boot

Cause

TCR granule mismatch:

- page tables built for 4 KB
- TCR configured for 16 KB or 64 KB

Fix

Ensure:

```
TCR_ELx.TG0/TG1 match page-table granule
```

****

****

27.7 — Failure Mode #6: Wrong Alignment

****

****

Symptom

- MMU enable crash
- page-table walk faults
- Stage-2 faults
- random memory corruption

Cause

Page tables not aligned to:

- 4 KB

- 16 KB
- 64 KB
- 4 KB * number of entries

Fix

Align page tables to granule size.

****

****

27.8 — Failure Mode #7: Wrong Block/Page Usage

****

****

Symptom

- unexpected fragmentation
- TLB pressure
- performance loss
- SMP inconsistencies

Cause

Using pages where blocks should be used.

Fix

Use blocks for:

- kernel
- guest RAM
- large contiguous regions

Use pages for:

- MMIO
- small regions
- per-process mappings

****

****

27.9 — Failure Mode #8: Stale TLB Entries

****

****

Symptom

- descriptor updates not visible
- random crashes
- inconsistent behavior across cores
- “ghost mappings”

Cause

Missing TLBI.

Fix

Correct sequence:

```
DSB SY
TLBI <operation>
DSB SY
ISB
```

****

****

27.10 — Failure Mode #9: Stale Walk-Cache Entries

****

****

Symptom

- descriptor updates ignored
- SMP cores see different mappings
- nondeterministic faults

Cause

Missing BBM (break-before-make).

Fix

Correct BBM sequence:

```
Clear old descriptor
DSB SY
TLBI
DSB SY
```

```
Write new descriptor
DSB SY
ISB
```

****

****

27.11 — Failure Mode #10: Missing DSB/ISB

****

****

Symptom

- unpredictable behavior
- stale translations
- stale device writes
- stale code

Cause

Missing barriers.

Fix

Use:

```
DSB SY
ISB
```

after MMU changes.

****

****

27.12 — Failure Mode #11: Incorrect Stage-2 Mapping

****

****

Symptom

- guest kernel cannot boot
- guest page faults
- guest MMIO failures
- VM crashes

Cause

Stage-2 descriptor wrong:

- wrong S2AP
- wrong S2PXN/S2UXN
- wrong S2AttrIndx
- wrong IPA alignment

Fix

Correct Stage-2 descriptor.

****

****

27.13 — Failure Mode #12: Incorrect Stage-2 Permissions

****

****

Symptom

- guest sees read-only memory
- guest cannot execute code
- guest cannot access MMIO

Cause

Hypervisor overrides guest permissions incorrectly.

Fix

Set correct S2AP/S2PXN/S2UXN.

****

****

27.14 — Failure Mode #13: Incorrect Device Mapping

****

****

Symptom

- device malfunction
- corrupted registers
- DMA corruption
- interrupts fail

Cause

MMIO mapped incorrectly.

Fix

Use:

```
AttrIndx = 2 (Device-nGnRnE)
SH = 00 (Non-Shareable)
PXN = 1
UXN = 1
```

****

****

27.15 — Failure Mode #14: Incorrect DMA Buffer Mapping

****

****

Symptom

- DMA reads stale data
- DMA writes not visible
- corrupted buffers

Cause

DMA buffers mapped incorrectly.

Fix

Use:

```
Normal WB or Normal NC
Inner Shareable
```

****

****

27.16 — Failure Mode #15: Incorrect Cache Maintenance

****

****

Symptom

- DMA corruption
- stale descriptors
- stale code
- stale device state

Cause

Missing DC operations.

Fix

Before DMA read:

```
DC  CVAC
DSB  SY
```

After DMA write:

```
DC  CIVAC
DSB  SY
```

****

****

27.17 — Failure Mode #16: Incorrect Interrupt Vector Mapping

****

****

Symptom

- interrupts crash
- exception entry fails
- CPU cannot fetch vector instructions

Cause

Vectors mapped as Device memory.

Fix

Map vectors as:

```
Normal WB
Inner Shareable
Executable
```

****

****

27.18 — Failure Mode #17: Incorrect GIC Mapping

****

****

Symptom

- interrupts fail
- GIC registers return garbage
- interrupt routing broken

Cause

GIC mapped incorrectly.

Fix

Use:

```
Device-nGnRnE
Non-Shareable
4 KB pages
```

****

****

27.19 — Failure Mode #18: Incorrect Boot Identity Mapping

****

****

Symptom

- MMU enable crash
- early boot failure
- PC jumps to unmapped address

Cause

Identity mapping missing.

Fix

Identity map:

- boot code
- stack
- MMIO
- exception vectors

****

****

27.20 — Chapter Summary

****

****

This chapter covered:

- wrong attributes
- wrong shareability
- wrong permissions
- missing AF
- wrong granule
- wrong alignment
- wrong block/page usage
- stale TLB
- stale walk-cache
- missing barriers
- Stage-2 mapping failures
- Stage-2 permission failures
- device mapping failures
- DMA mapping failures
- cache maintenance failures
- interrupt vector failures
- GIC mapping failures
- boot identity mapping failures